

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline – Customer Order Processing

-

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data cleaning, implements SCD Type 2 logic using Apache Hudi, and produces aggregated analytics outputs queryable via AWS Athena.

Scope

- In Scope:
 - Ingestion of customer and order CSV files from specified S3 paths
 - Data cleaning (null removal, deduplication)
 - Registration of datasets in AWS Glue Data Catalog
 - SCD Type 2 implementation for order summary using Apache Hudi
 - Incremental processing triggered by customer data changes
 - Generation of customer aggregate spend analytics table
 - Storage of curated and analytics data in S3
 - Queryability via AWS Athena
- Out of Scope:
 - Real-time streaming ingestion
 - Data quality profiling beyond specified cleaning rules
 - User interface or reporting layer
 - Data archival or retention policies
 - Security and access control implementation details

Assumptions

1. CSV files in source S3 paths follow the schemas defined in Section 2 without header variations
2. CustId is the primary key for customer data and is immutable
3. OrderId is the primary key for order data
4. Date fields in Order data are in a parseable format (assumed ISO 8601 or consistent format)
5. The Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists

6. AWS Glue job has appropriate IAM permissions to read from source S3 paths and write to target S3 paths
7. Incremental processing is triggered by the presence of new or updated customer files
8. "Null" (string) and NULL (actual null) are the only null representations requiring removal
9. Duplicate detection is based on all columns in each dataset
10. OpTs (operation timestamp) is system-generated at processing time
11. TotalAmount in aggregation is calculated as $\text{PricePerUnit} \times \text{Qty}$ summed per customer
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Ingest Customer CSV Files

- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a structured format for processing.

- Business Objective:

Enable downstream processing by making customer master data available in a queryable, structured format.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/customerdata/
- Format: CSV
- Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	CustId	String	Customer unique identifier	
	Name	String	Customer name	
	EmailId	String	Customer email address	
	Region	String	Customer geographic region	

- Processing Rules / Functional Steps:
 1. Read all CSV files from the source S3 path
 2. Parse CSV content using the defined schema
 3. Load data into a Spark DataFrame for further processing
- Outputs:

- In-memory customer dataset ready for cleaning
- Preconditions:
 - Source S3 path exists and contains at least one CSV file
 - CSV files conform to the defined schema
- Postconditions:
 - Customer data is loaded into memory without schema errors
- Error / Validation Conditions:
 - If source path is empty or inaccessible, log error and halt processing
 - If CSV schema does not match expected columns, log error and halt processing
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Ingest Order CSV Files
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a structured format for processing.

- Business Objective:

Enable order transaction analysis by making order data available in a queryable, structured format.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/orderdata/
 - Format: CSV
 - Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	OrderId	String	Order unique identifier	
	ItemName	String	Name of ordered item	
	PricePerUnit	Decimal	Price per unit of item	
	Qty	Integer	Quantity ordered	
	Date	Date	Order date	
	CustId	String	Foreign key to Customer	

- Processing Rules / Functional Steps:
 1. Read all CSV files from the source S3 path

2. Parse CSV content using the defined schema
3. Load data into a Spark DataFrame for further processing

- Outputs:
- In-memory order dataset ready for cleaning
- Preconditions:
- Source S3 path exists and contains at least one CSV file
- CSV files conform to the defined schema
- Postconditions:
- Order data is loaded into memory without schema errors
- Error / Validation Conditions:
- If source path is empty or inaccessible, log error and halt processing
- If CSV schema does not match expected columns, log error and halt processing
-

FR-CLEAN-001: Remove Null Values and Duplicates

- Title: Data Cleaning – Null Removal and Deduplication
- Description:

The system shall remove records containing "Null" (string) or NULL (actual null) values in any column, and remove duplicate records from both customer and order datasets.

- Business Objective:

Ensure data quality by eliminating incomplete and redundant records before downstream processing.

- Inputs:
- In-memory customer dataset (from FR-INGEST-001)
- In-memory order dataset (from FR-INGEST-002)
- Processing Rules / Functional Steps:
- 1. For customer dataset:
- Remove rows where any column contains "Null" (case-sensitive string) or NULL
- Remove duplicate rows based on all columns
- 2. For order dataset:
- Remove rows where any column contains "Null" (case-sensitive string) or NULL
- Remove duplicate rows based on all columns
- 3. Log count of removed records for each dataset

- Outputs:
- Cleaned customer dataset
- Cleaned order dataset
- Preconditions:
- Customer and order datasets are loaded in memory
- Postconditions:
- No records contain "Null" or NULL values
- No duplicate records exist in either dataset
- Error / Validation Conditions:
- If all records are removed due to null values, log warning and continue with empty dataset
-

FR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Title: Register Customer and Order Tables in Glue Catalog
- Description:

The system shall register the cleaned customer and order datasets as tables in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Enable SQL-based querying and discovery of source datasets through AWS Athena and other AWS analytics services.

- Inputs:
- Cleaned customer dataset (from FR-CLEAN-001)
- Cleaned order dataset (from FR-CLEAN-001)
- Processing Rules / Functional Steps:
- 1. Write cleaned customer data to Glue Catalog table:
 - Database: `gen\_ai\_poc\_databrickscoe`
 - Table: `sdlc\_wizard\_customer`
- 2. Write cleaned order data to Glue Catalog table:
 - Database: `gen\_ai\_poc\_databrickscoe`
 - Table: `sdlc\_wizard\_order`
- 3. Ensure table schemas match the input schemas defined in FR-INGEST-001 and FR-INGEST-002
- Outputs:

- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_customer`
- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_order`
- Preconditions:
 - Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists
 - Cleaned datasets are available
- Postconditions:
 - Tables are queryable via AWS Athena
 - Table schemas are registered in Glue Data Catalog
- Error / Validation Conditions:
 - If database does not exist, log error and halt processing
 - If table registration fails, log error and halt processing
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: SCD Type 2 Order Summary with Hudi
- Description:

The system shall join customer and order data, apply SCD Type 2 logic to track historical changes, and store the result in Apache Hudi format on S3. When customer data changes, new active rows shall be inserted and previous rows shall be closed with updated timestamps.

- Business Objective:

Maintain a complete historical record of order summaries that reflects changes in customer attributes over time, enabling point-in-time analysis.

- Inputs:
 - Cleaned customer dataset (from FR-CLEAN-001)
 - Cleaned order dataset (from FR-CLEAN-001)
 - Existing `ordersummary` Hudi table (for incremental updates)
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. For each joined record, determine if it represents a new or changed record compared to existing `ordersummary` table
 3. For new records:
 - Insert with IsActive = True
 - Set StartDate = current processing date

- Set EndDate = NULL
 - Set OpTs = current processing timestamp
4. For changed records (customer attributes updated):
- Close existing active row: set IsActive = False, EndDate = current processing date, update OpTs
 - Insert new active row with updated attributes: IsActive = True, StartDate = current processing date, EndDate = NULL, OpTs = current timestamp
5. Write results to Hudi table using upsert operation
6. Register or update Glue Catalog table `ordersummary` pointing to Hudi dataset
- Outputs:
 - Target Table: `ordersummary`
 - Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Format: Apache Hudi
 - Glue Catalog Table: `gen\_ai\_poc\_databrickscoe.ordersummary`
 - Schema (minimum required columns):
 - All columns from Customer schema (CustId, Name, EmailId, Region)
 - All columns from Order schema (OrderId, ItemName, PricePerUnit, Qty, Date)
 - IsActive (Boolean)
 - StartDate (Date)
 - EndDate (Date, nullable)
 - OpTs (Timestamp)
 - Preconditions:
 - Cleaned customer and order datasets are available
 - S3 target path is accessible
 - Hudi libraries are available in Glue job environment
 - Postconditions:
 - All current records have IsActive = True and EndDate = NULL
 - All historical records have IsActive = False and EndDate populated
 - Hudi table is queryable via Athena
 - Error / Validation Conditions:
 - If join produces no records, log warning and create empty Hudi table
 - If Hudi write fails, log error and halt processing
 - If customer record has no matching orders, exclude from ordersummary (inner join behavior assumed)

-

FR-INCR-001: Incremental Processing Based on Customer Updates

- Title: Trigger Incremental SCD2 Updates on Customer Changes

- Description:

The system shall detect changes in customer data and trigger incremental updates to the `ordersummary` table, applying SCD Type 2 logic only to affected records.

- Business Objective:

Optimize processing performance by updating only changed records rather than reprocessing the entire dataset, while maintaining historical accuracy.

- Inputs:

- New or updated customer data (from FR-INGEST-001)
- Existing `ordersummary` Hudi table

- Processing Rules / Functional Steps:

1. Identify customer records that are new or have changed attributes (Name, EmailId, or Region) compared to previous load

2. For changed customers:

- Retrieve all related order records from cleaned order dataset
- Apply SCD Type 2 logic as defined in FR-SCD2-001 for these records only

3. Perform Hudi upsert operation to update `ordersummary` table

4. Maintain IsActive, StartDate, EndDate, and OpTs columns as defined in FR-SCD2-001

- Outputs:

- Updated `ordersummary` Hudi table with incremental changes applied

- Preconditions:

- Customer data has been ingested and cleaned
- Existing `ordersummary` table is accessible

- Postconditions:

- Only affected records are updated in `ordersummary`
- SCD Type 2 integrity is maintained across all records

- Error / Validation Conditions:

- If no customer changes are detected, log informational message and skip SCD2 processing
- If incremental update fails, log error and halt processing

•

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table
- Description:

The system shall calculate total spending per customer by aggregating order data and store the results in a dedicated analytics table for reporting and analysis.

- Business Objective:

Provide a pre-aggregated view of customer spending to enable fast analytical queries and business intelligence reporting.

- Inputs:
 - Cleaned customer dataset (from FR-CLEAN-001)
 - Cleaned order dataset (from FR-CLEAN-001)
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. Calculate TotalAmount for each order as: $\text{PricePerUnit} \times \text{Qty}$
 3. Group by customer Name and Date
 4. Sum TotalAmount for each group
 5. Write aggregated results to S3 in Parquet format
 6. Register or update Glue Catalog table `customeragggregatespend`
- Outputs:
 - Target Table: `customeragggregatespend`
 - Storage Location: s3://adif-sdlc/analytics/customeragggregatespend/
 - Format: Parquet
 - Glue Catalog Table: `gen\_ai\_poc\_databrickscoe.customeragggregatespend`
 - Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	Name	String	Customer name	
	TotalAmount	Decimal	Sum of ($\text{PricePerUnit} \times \text{Qty}$)	
	Date	Date	Order date	

- Preconditions:
 - Cleaned customer and order datasets are available

- S3 target path is accessible
- Postconditions:
 - Aggregated data is available for querying via Athena
 - Table contains one row per customer per date with total spending
- Error / Validation Conditions:
 - If join produces no records, create empty table and log warning
 - If aggregation fails, log error and halt processing
 - If customer has no orders, exclude from aggregate table
-

3. Non-Functional Considerations

Performance

- Processing should leverage Spark's distributed computing capabilities for scalability
- Incremental processing (FR-INCR-001) should significantly reduce processing time compared to full reprocessing

Monitoring and Observability

- AWS CloudWatch shall be used to monitor Glue job execution metrics
- Job failures, warnings, and record counts should be logged to CloudWatch Logs

Data Governance

- AWS Lake Formation may optionally be used for fine-grained access control and data governance
- All data catalog tables should be discoverable through AWS Glue Data Catalog

Query Performance

- Athena shall be used as the primary query engine for all registered tables
- Hudi and Parquet formats are chosen to optimize query performance

Storage Architecture

- Raw/Source Layer: s3://adif-sdlc/sdlc_wizard/ (customer and order data)
- Curated Layer: s3://adif-sdlc/curated/sdlc_wizard/ (SCD2 order summary)
- Analytics Layer: s3://adif-sdlc/analytics/ (aggregated customer spend)
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description
----------------	-------------	-------------

-----	-----	-----
FR-INGEST-001	Source Input (1), Schemas (2)	Customer CSV ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/
FR-INGEST-002	Source Input (1), Schemas (2)	Order CSV ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/
FR-CLEAN-001	Data Cleaning (3)	Null and duplicate removal
FR-CATALOG-001	Glue Catalog Tables (4)	Registration of sdlc_wizard_customer and sdlc_wizard_order
FR-SCD2-001	SCD Type 2 (5), Glue Catalog Tables (4)	Hudi-based SCD2 implementation for ordersummary
FR-INCR-001	Incremental Logic (6)	Customer change-driven incremental updates
FR-AGG-001	Aggregation Table (7), Glue Catalog Tables (4)	Customer aggregate spend calculation and storage

- Traceability to AWS Architecture Components (Section 8):
- Storage (S3): All FR requirements use S3 for input/output
- ETL (AWS Glue with Spark + Hudi): FR-CLEAN-001, FR-SCD2-001, FR-INCR-001, FR-AGG-001
- Catalog (Glue Data Catalog): FR-CATALOG-001, FR-SCD2-001, FR-AGG-001
- Query (Athena): Implicit in all catalog registration requirements
- Monitoring (CloudWatch): Non-functional consideration
- Lake Formation: Optional governance layer (non-functional consideration)
-
- Document Version: 1.0
- Date: 2025-01-29
- Status: Final